CO-3-ASSIGNMENT

Course Outcome: CO3 – Design Divide and Conquer algorithms for Merge Sort, Quick Sort, Binary Search and Matrix Multiplication

CO3 Coverage: DIVIDE AND CONQUER: Introduction- Finding the maximum and minimum-Merge sort-Quick sort-Binary Search-Stassen's Matrix Multiplication-Karatsuba Algorithm for Multiplication-Closest Pair of Points-Median of Medians-Meet In Middle Technique- Advantages, and Disadvantages of Divide and Conquer.

Course Outcome	CO3
Assignment	Assignment tool
Faculty	Dr. F. Mary Harin Fernandez
Student Name	Pooja Sree M
Register Number	192521403
Course Code	CSA0608
Course Name	Design and Analysis of Algorithm

1. Introduction to Divide and Conquer

Divide and Conquer is an algorithm design technique that solves a problem by dividing it into smaller subproblems of the same or related type, solving those subproblems recursively, and combining their results. A correct design identifies a base case, a decomposition rule, the information returned by each recursive call, and a combine operation. Divide and Conquer is one of the most important and widely used techniques in algorithm design. It is a problem-solving approach that helps solve complex problems by breaking them into smaller and simpler subproblems. Instead of solving the entire problem at once, the technique divides the main problem into manageable parts, solves each part independently, and combines the results to obtain the final solution.

The main idea behind Divide and Conquer is to reduce the complexity of a problem and make the solution more efficient. It mainly follows three important steps: Divide, Conquer, and Combine.

1. **Divide:** The original problem is divided into two or more smaller subproblems of the same type.
2. **Conquer:** Each subproblem is solved independently, usually using recursion. If a subproblem is small enough, it is solved directly.
3. **Combine:** The solutions obtained from the smaller subproblems are combined to produce the final result.

This technique is commonly used in computer science because it can reduce the time required to solve large problems. Many efficient algorithms are developed using this approach. It is especially useful when a problem can be divided into independent parts and the results can be combined easily.

Divide and Conquer also helps programmers understand the structure of a problem clearly. By solving smaller parts separately, the algorithm becomes easier to design, implement, test, and analyse.

Recursion plays an important role in this technique because the same process is repeatedly applied to smaller versions of the original problem.

Different algorithms based on the Divide and Conquer technique are studied and implemented. These include finding Maximum and Minimum elements, Binary Search, Merge Sort, Quick Sort, Karatsuba Multiplication, Strassen's Matrix Multiplication, Closest Pair of Points, Median of Medians, and Meet-in-the-Middle techniques.

Performance analysis is also an important part of this assignment. The algorithms are evaluated based on their execution time, number of operations, and efficiency for different datasets. Tables and graphs can be used to represent the experimental results and compare the performance of different approaches.

1.1 General design pattern

```
D AND C(problem):
    if problem is small enough:
        return solve_directly(problem)
    divide problem into smaller subproblems
    solve each subproblem recursively
    combine the returned results
    return combined result
```

1.2 Recurrence and complexity

A common recurrence is $T(n) = aT(n/b) + f(n)$, where a is the number of recursive subproblems, n/b is the approximate size of each subproblem, and f(n) is the work for dividing and combining. The recurrence is solved using a recursion tree, substitution, or the Master Theorem when its conditions apply.

- Advantages: often reduces a large search space, supports clear recursive structure, can expose parallelism, and may improve asymptotic time.
- Disadvantages: recursive calls add stack and function-call overhead; combining results can be expensive; extra memory may be required; unbalanced decomposition can remove the expected benefit.
- Good candidates have manageable base cases, substantially smaller subproblems, and an efficient combine step.

Q1 – Decomposition and Elimination in Ordered Data

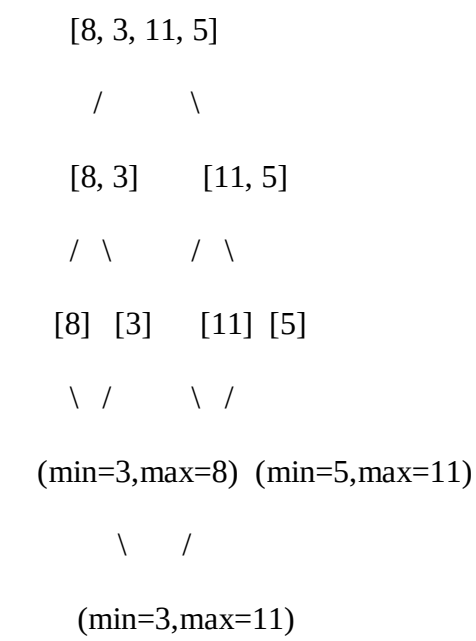
Part A – Finding Maximum and Minimum

Goal: find both the largest and smallest elements of an array using a Divide-and-Conquer/tournament strategy. Each recursive call returns a pair (minimum, maximum).

Algorithm design and decomposition

- Base case: for one element, that element is both minimum and maximum. For two elements, compare them once and return them in the correct order.
- Divide: split the array into left and right halves.
- Conquer: recursively find the minimum and maximum in each half.
- Combine: compare the two local minima once and the two local maxima once.

Decomposition example for A = [8, 3, 11, 5]:



Pseudocode:

```
MAXMIN(A, low, high):
    if low == high:
        return (A[low], A[low])
    if high == low + 1:
        if A[low] < A[high]:
            return (A[low], A[high])
        else:
            return (A[high], A[low])

    mid = floor((low + high) / 2)
    (minL, maxL) = MAXMIN(A, low, mid)
    (minR, maxR) = MAXMIN(A, mid + 1, high)
    return (min(minL, minR), max(maxL, maxR))
```

Python implementation with comparison counting:

```
def maxmin_dc(a):

    comparisons = 0

    def solve(lo, hi):

        nonlocal comparisons

        if lo == hi:
            return a[lo], a[lo]

        if hi == lo + 1:
            comparisons += 1
            return (a[lo], a[hi]) if a[lo] < a[hi] else (a[hi], a[lo])

        mid = (lo + hi) // 2

        mn1, mx1 = solve(lo, mid)
        mn2, mx2 = solve(mid + 1, hi)

        comparisons += 2

        return min(mn1, mn2), max(mx1, mx2)

    result = solve(0, len(a) - 1)

    return result[0], result[1], comparisons

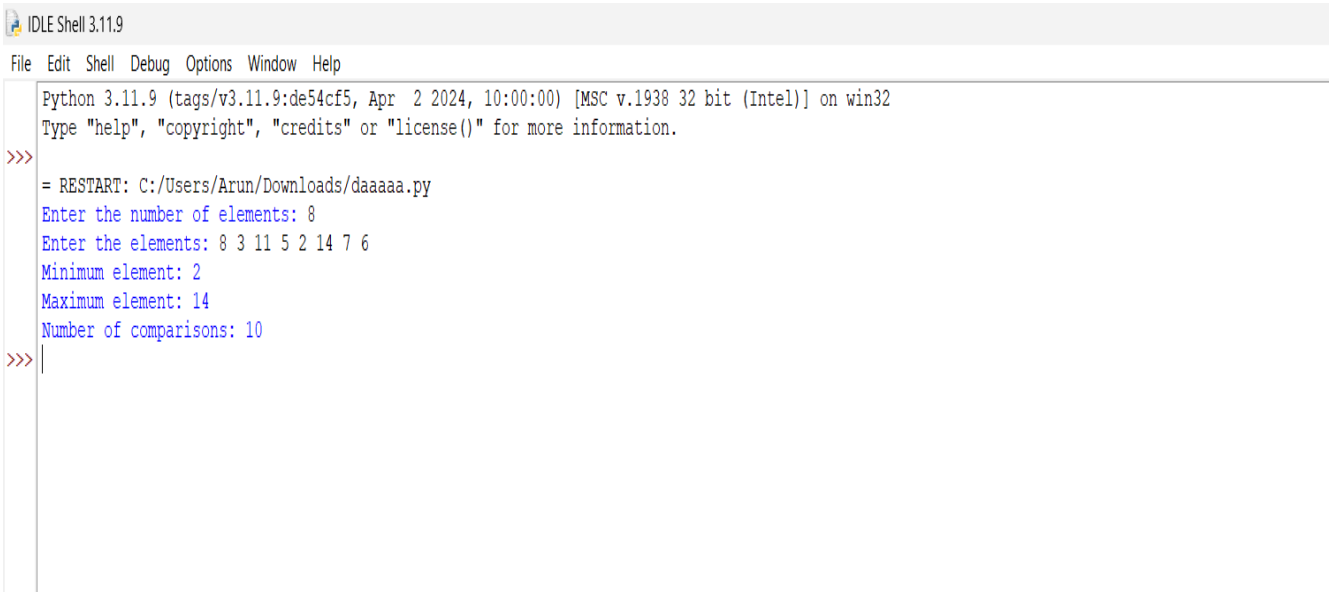

# Getting input from the user
n = int(input("Enter the number of elements: "))

if n <= 0:
    print("Please enter a valid number of elements.")
else:
    data = list(map(int, input("Enter the elements: ").split()))

    if len(data) != n:
        print("Please enter exactly", n, "elements.")
    else:
        minimum, maximum, comparisons = maxmin_dc(data)
```

```
    print("Minimum element:", minimum)
    print("Maximum element:", maximum)
print("Number of comparisons:", comparisons)
```

OUTPUT:



Recurrence and mathematical analysis:

For n = 1, T(1) = 0. For n = 2, one comparison is used. For even n, the recurrence for the comparison count is C(n) = 2C(n/2) + 2. For n a power of two, this gives C(n) = 3n/2 − 2. Thus the running time is Θ(n), and the tournament method uses fewer comparisons than independently scanning for the minimum and maximum.

Method	Maximum comparisons (n elements)	Time
Independent min and max scans	2n − 2	Θ(n)
Pairwise / tournament method (even n)	3n/2 − 2	Θ(n)

For n = 8, independent scans use up to 14 comparisons, while the divide-and-conquer tournament count is 10. For odd or non-power-of-two input sizes, exact counts depend on how the split and base cases are handled.

Experimental dataset and results

Input size	Example dataset pattern	Expected D&C comparisons (power-of-two n)	Independent scans upper bound
8	[8,3,11,5,2,14,7,6]	10	14
16	Increasing integers 1…16	22	30
32	Increasing integers 1…32	46	62
64	Increasing integers 1…64	94	126

Observation: the comparison count grows linearly. The exact D&C count is deterministic for a fixed recursive split and base-case policy; it does not depend on whether the data is increasing, decreasing, or random.

Part B – Binary Search

Binary Search finds a target in a sorted array. It compares the target with the middle element and discards the half in which the target cannot occur.

Decision rule, elimination logic and termination

- If target equals the middle value, return its index.
- If target is smaller, continue only in the left half; every value in the right half is at least the middle value and therefore cannot be the target.
- If target is larger, continue only in the right half; every value in the left half is at most the middle value and therefore cannot be the target.
- Terminate successfully when the middle value matches, or unsuccessfully when low > high.

Pseudocode

```
BINARY-SEARCH(A, target):
    low = 0
    high = length(A) - 1
    while low <= high:
        mid = floor((low + high) / 2)
        if A[mid] == target:
            return mid
        else if target < A[mid]:
            high = mid - 1
        else:
            low = mid + 1
    return NOT-FOUND
```

Python implementation with comparison count

```
def binary_search(a, target):
    low, high, comparisons = 0, len(a) - 1, 0
```

```
    while low <= high:

        mid = (low + high) // 2

        comparisons += 1

        if a[mid] == target:
            return mid, comparisons

        comparisons += 1

        if target < a[mid]:
            high = mid - 1
        else:
            low = mid + 1

    return -1, comparisons


# Getting input from the user
n = int(input("Enter the number of elements: "))

if n <= 0:
    print("Please enter a valid number of elements.")
else:
    values = list(map(int, input("Enter the sorted elements: ").split()))

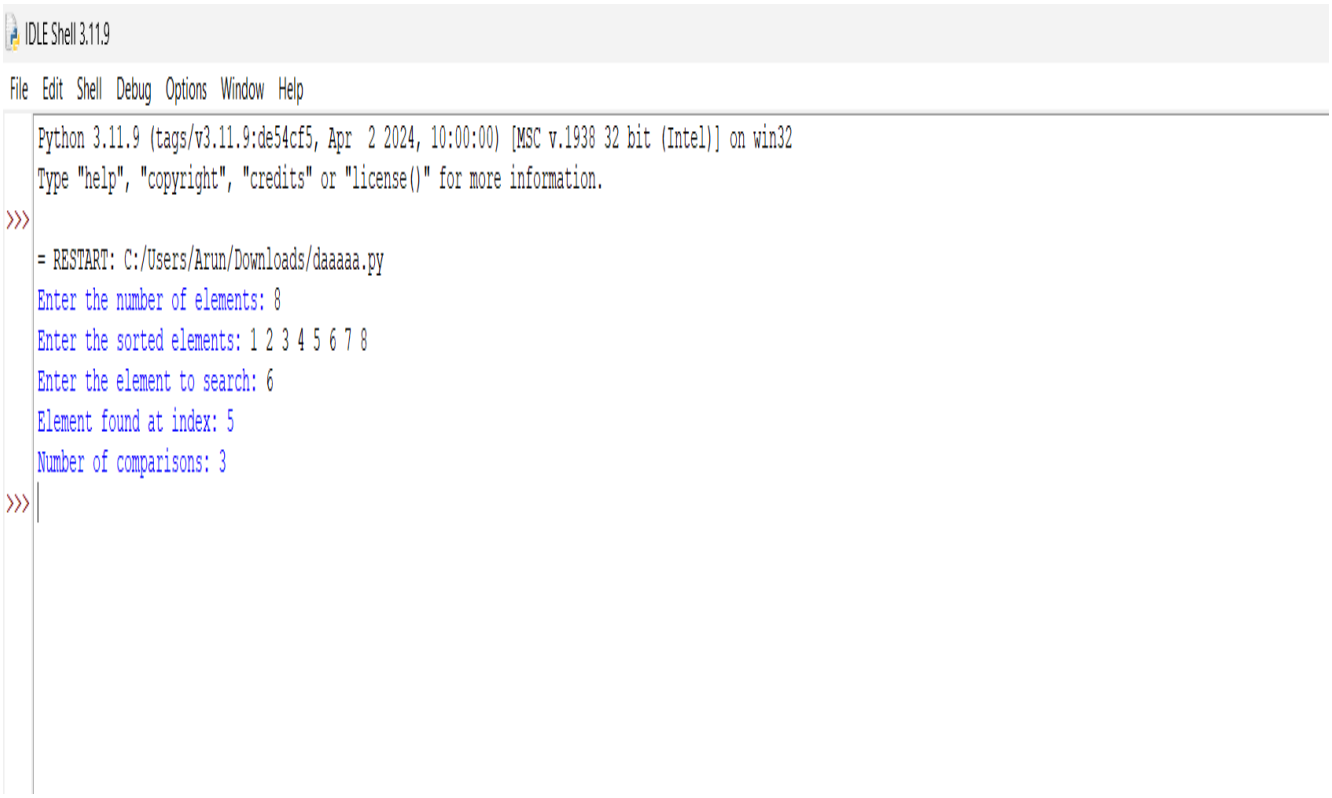
    if len(values) != n:
        print("Please enter exactly", n, "elements.")
    elif values != sorted(values):
        print("Please enter elements in sorted order.")
    else:
        target = int(input("Enter the element to search: "))

        index, comparisons = binary_search(values, target)

        if index != -1:
            print("Element found at index:", index)
        else:
            print("Element not found.")
```

```
print("Number of comparisons:", comparisons)
```

OUTPUT:



The code counts one equality test and, when needed, one ordering test per loop. State the counting convention when comparing results. The number of iterations is O(log n); best case is O(1), and worst/average case is O(log n).

For successful and unsuccessful experiments, use sorted arrays of sizes 16, 32, 64, 128, 256 and 512. Test targets at the first, middle and last positions, plus values below, between and above the array values. Record index or NOT-FOUND and the comparison counter.

n	**Approximate worst-case iterations**	**Growth description**
16	5	logarithmic

32	6	logarithmic
64	7	logarithmic
128	8	logarithmic
256	9	logarithmic
512	10	logarithmic

Conclusion:

The Max-Min algorithm and Binary Search demonstrate the effective application of the Divide and Conquer technique in searching and selection problems. The Max-Min algorithm divides the given array into smaller subarrays and independently identifies the minimum and maximum elements. The results from these subarrays are then combined to obtain the overall minimum and maximum values. This method reduces unnecessary comparisons and improves the efficiency of finding extreme elements. Binary Search works by repeatedly dividing a sorted array into two halves and comparing the target value with the middle element. Based on the comparison, the search continues only in the required half, reducing the search space at every step. The Max-Min algorithm has a time complexity of $\Theta(n)$, while Binary Search achieves $\Theta(\log n)$ time complexity. The implementation and analysis of these algorithms provide a clear understanding of recursion, problem decomposition, comparison reduction, and efficient searching techniques.

Q2 – The Quality of Decomposition in Sorting

➢ Merge Sort

Merge Sort divides the array into two approximately equal halves, recursively sorts each half, and merges the two sorted halves. Its decomposition is balanced regardless of the original order of the input.

Pseudocode:

```
MERGE-SORT(A):
    if length(A) <= 1:
```

```
    return A

  split A into left and right halves

  left_sorted = MERGE-SORT(left)

  right_sorted = MERGE-SORT(right)

  return MERGE(left_sorted, right_sorted)


MERGE(L, R):

  create empty result

  compare the first unused item of L and R

  append the smaller item; advance that input

  append all remaining items

  return result
```

Python implementation:

```
def merge_sort(a):

    if len(a) <= 1:
        return a[:]

    mid = len(a) // 2

    left = merge_sort(a[:mid])
    right = merge_sort(a[mid:])

    i = j = 0
    out = []

    while i < len(left) and j < len(right):

        if left[i] <= right[j]:
            out.append(left[i])
            i += 1
        else:
```

```
            out.append(right[j])
            j += 1

    return out + left[i:] + right[j:]


# Getting input from the user
n = int(input("Enter the number of elements: "))

if n <= 0:
    print("Please enter a valid number of elements.")
else:
    data = list(map(int, input("Enter the elements: ").split()))

    if len(data) != n:
        print("Please enter exactly", n, "elements.")
    else:
        sorted_data = merge_sort(data)

        print("Original array:", data)
    print("Sorted array:", sorted_data)
```

OUTPUT:



Recurrence and complexity:

$T(n) = 2T(n/2) + \Theta(n)$, because two halves are sorted recursively and merging takes linear time. By the Master Theorem, $T(n) = \Theta(n \log n)$. Auxiliary space is $\Theta(n)$ for the merge buffer in this implementation.

➢ Quick Sort

Quick Sort selects a pivot, partitions the array into values smaller than or equal to the pivot and values greater than the pivot, and recursively sorts the partitions. This report uses the last element as the pivot so the experiment is reproducible.

Pseudocode:

```
QUICK-SORT(A, low, high):
    if low < high:
        p = PARTITION(A, low, high)  // last element is pivot
        QUICK-SORT(A, low, p - 1)
        QUICK-SORT(A, p + 1, high)

PARTITION(A, low, high):
    pivot = A[high]
    i = low - 1
    for j from low to high - 1:
        if A[j] <= pivot:
            i = i + 1
            swap A[i], A[j]
    swap A[i + 1], A[high]
    return i + 1
```

Python implementation with partition metrics:

```
def quick_sort_metrics(a):

    a = a[:]
```

```
    partitions, max_depth = [], 0

    def sort(lo, hi, depth):

        nonlocal max_depth

        max_depth = max(max_depth, depth)

        if lo < hi:

            pivot = a[hi]

            i = lo - 1

            for j in range(lo, hi):

                if a[j] <= pivot:
                    i += 1
                    a[i], a[j] = a[j], a[i]

            p = i + 1

            a[p], a[hi] = a[hi], a[p]

            partitions.append((p - lo, hi - p))

            sort(lo, p - 1, depth + 1)
            sort(p + 1, hi, depth + 1)

    sort(0, len(a) - 1, 1)

    return a, partitions, max_depth

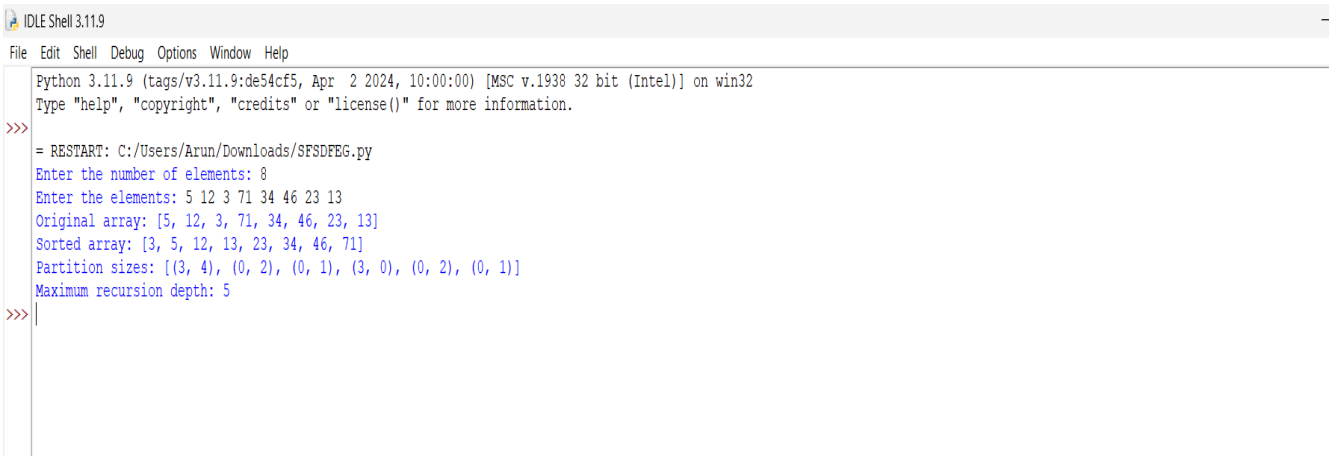

# Getting input from the user
n = int(input("Enter the number of elements: "))

if n <= 0:
    print("Please enter a valid number of elements.")
else:
    data = list(map(int, input("Enter the elements: ").split()))
```

```
if len(data) != n:
    print("Please enter exactly", n, "elements.")
else:
    sorted_data, partitions, max_depth = quick_sort_metrics(data)

    print("Original array:", data)
    print("Sorted array:", sorted_data)
    print("Partition sizes:", partitions)
print("Maximum recursion depth:", max_depth)
```

OUTPUT:



Recurrence and complexity:

Best/balanced case: $T(n) = 2T(n/2) + \Theta(n) = \Theta(n \log n)$. Worst case: $T(n) = T(n-1) + \Theta(n) = \Theta(n^2)$, for example when the chosen pivot repeatedly becomes the smallest or largest element. Average expected time with a suitably randomized pivot is $\Theta(n \log n)$, while worst-case stack depth can be $\Theta(n)$.

Balanced and unbalanced experiment design:

Scenario	Suggested input (last-element pivot)	Expected partition behaviour	Expected recursion depth
Balanced-like	A shuffled permutation; repeat with fixed random	Partitions near equal sizes on average	O(log n) typical

	seeds		
Severely unbalanced	Already sorted ascending or descending data	One side has 0 elements and other has $n-1$ for this pivot rule	O(n)

Record each partition as (left_size, right_size), maximum recursion depth, elapsed time, and correctness. Repeat for n = 100, 500, 1,000, 2,000 and 4,000. A last-element pivot on sorted ascending input produces the worst-case pattern; a randomized or median-of-three pivot can reduce the likelihood of this pattern.

Merge Sort versus Quick Sort

Feature	Merge Sort	Quick Sort
Decomposition	Always splits by position into near-equal halves	Depends on pivot and input values
Recurrence	$2T(n/2)+\Theta(n)$	$T(k)+T(n-k-1)+\Theta(n)$
Worst-case time	$\Theta(n \log n)$	$\Theta(n^2)$
Typical time	$\Theta(n \log n)$	$\Theta(n \log n)$
Extra memory	$\Theta(n)$ for this array implementation	Usually in-place partition; recursion stack
Predictability	Predictable recursion-tree height	Partition quality varies

Joint design question

Before choosing a sorting component when input characteristics are unknown, we would want to know the expected input size, whether data is already or nearly sorted, the distribution and frequency of

duplicates, whether stable ordering is required, available memory, worst-case latency requirements, and whether inputs may be adversarial. Merge Sort is a strong choice when predictable O(n log n) worst-case time and stability are important. Quick Sort can be attractive when in-place sorting and strong average performance are priorities, especially with randomized pivot selection or an introspective fallback.

Conclusion:

Merge Sort and Quick Sort demonstrate how the Divide and Conquer strategy can be used to arrange data efficiently. Merge Sort recursively divides the input array into smaller subarrays until each subarray contains a single element. These subarrays are then merged in sorted order to produce the final result. It provides consistent performance with a time complexity of $\Theta(n \log n)$ in the best, average, and worst cases. Quick Sort selects a pivot element and partitions the array into smaller and larger elements. The same process is recursively applied to the partitions until the complete array becomes sorted. Its performance depends on the selection of the pivot and the balance of the partitions. Balanced partitions provide an average time complexity of $\Theta(n \log n)$, whereas highly unbalanced partitions can result in a worst-case complexity of $\Theta(n^2)$. Merge Sort requires additional memory for merging, while Quick Sort mainly performs partitioning within the original array. The implementation and comparison of both algorithms help us understand their working principles, advantages, limitations, and performance under different input conditions.

Q3 – Breaking the Quadratic Barrier: Karatsuba Multiplication

Karatsuba multiplies large integers by splitting each number into high and low parts and reducing four half-size multiplications to three.

Decomposition and derivation

Let x = a·B + b and y = c·B + d, where B = 10^m (or an appropriate power of the numeric base) and a,b,c,d are the high/low parts. Then xy = ac·B² + (ad+bc)·B + bd. Compute z2 = ac, z0 = bd, and z1 = (a+b)(c+d) − z2 − z0. The product is z2·B² + z1·B + z0.

Pseudocode

```
KARATSUBA(x, y):
    if x or y is small:
        return x * y
    m = ceil(max(number_of_digits(x), number_of_digits(y)) / 2)
    B = 10^m
    a, b = divmod(x, B)
    c, d = divmod(y, B)
    z2 = KARATSUBA(a, c)
    z0 = KARATSUBA(b, d)
    z1 = KARATSUBA(a+b, c+d) - z2 - z0
    return z2*B*B + z1*B + z0
```

Python implementation and standard baseline

```
def standard_multiply(x, y):

    return x * y

def karatsuba(x, y, threshold=32):

    if x < 0 or y < 0:
        sign = -1 if (x < 0) != (y < 0) else 1
        return sign * karatsuba(abs(x), abs(y), threshold)
```

```
    if x < 10**threshold or y < 10**threshold:
        return x * y

    m = max(len(str(x)), len(str(y))) // 2

    B = 10**m

    a, b = divmod(x, B)
    c, d = divmod(y, B)

    z2 = karatsuba(a, c, threshold)
    z0 = karatsuba(b, d, threshold)
    z1 = karatsuba(a + b, c + d, threshold) - z2 - z0

    return z2 * B * B + z1 * B + z0


# Getting input from the user
x = int(input("Enter the first number: "))
y = int(input("Enter the second number: "))

threshold = int(input("Enter the threshold value (default 32): "))

if threshold <= 0:
    print("Please enter a positive threshold value.")
else:
    result1 = standard_multiply(x, y)
    result2 = karatsuba(x, y, threshold)

    print("Standard Multiplication Result:", result1)
    print("Karatsuba Multiplication Result:", result2)

    if result1 == result2:
        print("Both multiplication results are equal.")
    else:
        print("The multiplication results are not equal.")
```

OUTPUT:



The standard Python integer multiplication operator is a baseline, not necessarily the elementary school $O(n^2)$ algorithm; the Python runtime may already use optimized multiplication internally. For a fair algorithmic comparison, implement grade-school multiplication explicitly or state the runtime's built-in implementation as the baseline.

Recurrence and solution

Karatsuba performs three recursive multiplications on roughly n/2-digit numbers and linear work for additions, subtraction and splitting: $T(n) = 3T(n/2) + \Theta(n)$. The Master Theorem gives $T(n) = \Theta(n^{(\log_2 3)}) \approx \Theta(n^{1.585})$. Conventional grade-school multiplication takes $\Theta(n^2)$ single-digit operations.

Crossover experiment

Use operands with 32, 64, 128, 256, 512, 1,024 and 2,048 decimal digits. Generate reproducible random decimal strings without leading zeros. For each size, verify the two products match, repeat each timing at least 5–10 times, and report the median. Record recursive multiplication calls or a

counter if you instrument the implementation. Determine the crossover from the measured data; it depends on language, hardware and threshold.

Operand digits	Standard multiplication time	Karatsuba time	Winner
32	Measure locally	Measure locally	Do not assume
64	Measure locally	Measure locally	Do not assume
128	Measure locally	Measure locally	Do not assume
256	Measure locally	Measure locally	Do not assume
512	Measure locally	Measure locally	Do not assume
1,024	Measure locally	Measure locally	Do not assume

Why Karatsuba may be slower for small inputs: recursion, digit splitting, additions, temporary integer creation and function-call overhead can exceed the savings from reducing one multiplication. The implementation's threshold should be selected experimentally.

Joint conclusion

Karatsuba Multiplication demonstrates how mathematical decomposition can improve the efficiency of multiplying large integers. The conventional recursive multiplication approach requires four smaller multiplications when two numbers are divided into higher and lower parts. Karatsuba reduces these recursive multiplications from four to three by calculating three intermediate products and combining them using mathematical operations. Its recurrence relation is $T(n) = 3T(n/2) + O(n)$, resulting in an approximate time complexity of $\Theta(n^{1.585})$. This is theoretically more efficient than the conventional recursive multiplication approach. The implementation compares the result of Karatsuba multiplication with standard multiplication to verify the correctness of the output. Although the algorithm reduces the number of recursive multiplications, additional calculations and recursion overhead can affect its

practical performance for smaller inputs. It becomes particularly useful when multiplying sufficiently large integers. This algorithm provides an understanding of how reducing expensive operations through mathematical techniques can improve computational efficiency.

Q4 – The Reality of Strassen's Matrix Multiplication

Conventional matrix multiplication for two n×n matrices performs eight block multiplications when each matrix is divided into four equal blocks. Strassen's method reorganizes the algebra so that only seven block multiplications are needed, at the cost of additional additions and subtractions.

Conventional block decomposition

For A = [[A11,A12],[A21,A22]] and B = [[B11,B12],[B21,B22]], conventional multiplication is C11=A11B11+A12B21; C12=A11B12+A12B22; C21=A21B11+A22B21; C22=A21B12+A22B22. This uses eight half-size block multiplications.

Strassen's seven products

M1 = (A11 + A22)(B11 + B22)

M2 = (A21 + A22)B11

M3 = A11(B12 - B22)

M4 = A22(B21 - B11)

M5 = (A11 + A12)B22

M6 = (A21 - A11)(B11 + B12)

M7 = (A12 - A22)(B21 + B22)

C11 = M1 + M4 - M5 + M7

C12 = M3 + M5

C21 = M2 + M4

C22 = M1 - M2 + M3 + M6

Correctness follows by expanding each Mi using distributivity and cancelling terms. For example, C11 expands to A11B11 + A12B21; the other reconstructed blocks similarly simplify to the conventional block expressions.

Pseudocode:

```
STRASSEN(A, B):
    if matrix size is below threshold:
        return conventional_multiply(A, B)
    if size is odd or not a power of two:
        pad A and B with zero rows/columns to a suitable size
    split A and B into four blocks
    compute M1 through M7
    reconstruct C11, C12, C21, C22
    join the four C blocks
    remove padding and return the original-size result
```

Python implementation (square matrices; zero-padding):

```
def conventional(A, B):
    n = len(A)
    return [[sum(A[i][k] * B[k][j] for k in range(n))
             for j in range(n)] for i in range(n)]
def add(X, Y):
```

```
    n = len(X)
    return [[X[i][j] + Y[i][j] for j in range(n)] for i in range(n)]


def sub(X, Y):
    n = len(X)
    return [[X[i][j] - Y[i][j] for j in range(n)] for i in range(n)]
def strassen(A, B, threshold=2):
    n = len(A)
    if n <= threshold:
        return conventional(A, B)
    if n % 2:
        raise ValueError("Pad matrix to an even size before recursion")
    m = n // 2
    def split(X):
        return ([row[:m] for row in X[:m]],
                [row[m:] for row in X[:m]],
                [row[:m] for row in X[m:]],
                [row[m:] for row in X[m:]])
    A11, A12, A21, A22 = split(A)
    B11, B12, B21, B22 = split(B)
    M1 = strassen(add(A11, A22), add(B11, B22), threshold)
    M2 = strassen(add(A21, A22), B11, threshold)
    M3 = strassen(A11, sub(B12, B22), threshold)
```

```
    M4 = strassen(A22, sub(B21, B11), threshold)
    M5 = strassen(add(A11, A12), B22, threshold)
    M6 = strassen(sub(A21, A11), add(B11, B12), threshold)
    M7 = strassen(sub(A12, A22), add(B21, B22), threshold)

    C11 = add(sub(add(M1, M4), M5), M7)
    C12 = add(M3, M5)
    C21 = add(M2, M4)
    C22 = add(sub(add(M1, M3), M2), M6)
    return [C11[i] + C12[i] for i in range(m)] + \
           [C21[i] + C22[i] for i in range(m)]
def strassen_padded(A, B, threshold=2):
    n, m = len(A), 1
    while m < n:
        m *= 2
    Ap = [row[:] + [0] * (m - n) for row in A] + \
         [[0] * m for _ in range(m - n)]
    Bp = [row[:] + [0] * (m - n) for row in B] + \
         [[0] * m for _ in range(m - n)]
    Cp = strassen(Ap, Bp, threshold)
    return [row[:n] for row in Cp[:n]]
# Getting input from the user
n = int(input("Enter the size of the square matrices: "))
```

```
if n <= 0:
    print("Please enter a valid matrix size.")
else:
    print("Enter the elements of Matrix A row by row:")
    A = []
    for i in range(n):
        row = list(map(int, input().split()))
        A.append(row)
    print("Enter the elements of Matrix B row by row:")
    B = []
    for i in range(n):
        row = list(map(int, input().split()))
        B.append(row)
    if any(len(row) != n for row in A + B):
        print("Each row must contain exactly", n, "elements.")
    else:
        result1 = conventional(A, B)
        result2 = strassen_padded(A, B)
        print("Conventional Multiplication Result:")
        for row in result1:
            print(row)
        print("Strassen Multiplication Result:")
        for row in result2:
```

```
        print(row)

    print("Results are equal:", result1 == result2)
```

OUTPUT:



Recurrence, padding and engineering analysis:

$T(n) = 7T(n/2) + \Theta(n^2)$, giving $\Theta(n^{\wedge}(\log_2 7)) \approx \Theta(n^{\wedge}2.807)$. Conventional multiplication is $\Theta(n^3)$. Padding a non-power-of-two dimension to the next power of two can increase work and memory; for example, a 5×5 matrix is padded to 8×8 in the sample implementation. The code above handles square matrices by padding; it assumes both inputs are square and of equal dimensions.

- Seven recursive products reduce multiplication work, but Strassen performs more additions and subtractions.
- Recursive submatrices and temporary matrices increase memory traffic and allocation overhead.
- Cache locality and memory bandwidth can affect the practical crossover.

- Use conventional multiplication for small matrices, memory-constrained settings, or when numerical/implementation simplicity is more important than asymptotic improvement.

Experiment plan and results table

Test sizes 2, 4, 8, 16, 32, 64, 128 and 256, plus non-power-of-two sizes such as 5, 17 and 70. Use deterministic random integer matrices, compare outputs exactly, repeat timings, and record scalar multiplication counts. With the simple recursive Strassen method, the recursive multiplication count is 7^k for padded size 2^k; conventional multiplication performs n^3 scalar multiplications.

Matrix size	Conventional scalar multiplications	Strassen recursive scalar products (padded power of 2)	Measured time
8×8	512	343	Run and record
16×16	4,096	2,401	Run and record
32×32	32,768	16,807	Run and record
64×64	262,144	117,649	Run and record
128×128	2,097,152	823,543	Run and record

Joint engineering decision

Use conventional multiplication when matrices are small, when padding overhead is large, when memory is limited, or when the measured workload does not reach the Strassen crossover. Strassen is more attractive for sufficiently large dense matrices when implementation and memory costs are acceptable.

Conclusion:

Strassen's Matrix Multiplication demonstrates an advanced application of Divide and Conquer for performing matrix multiplication efficiently. Conventional matrix multiplication requires eight recursive block multiplications when matrices are divided into four smaller submatrices. Strassen's algorithm reduces these eight multiplications to seven by using mathematical additions and subtractions. The input matrices are divided into smaller blocks, and seven intermediate products are calculated. These products are then combined to generate the final result matrix. Its recurrence relation is $T(n) = 7T(n/2) + O(n^2)$, giving an approximate time complexity of $\Theta(n^{2.807})$, which is theoretically better than the $\Theta(n^3)$ complexity of conventional matrix multiplication. The implementation also handles matrix dimensions that are not powers of two by padding the matrices with zeros and removing the extra rows and columns after multiplication. The results are compared with conventional matrix multiplication to verify correctness. However, additional matrix additions, memory requirements, and padding operations can influence practical performance. This algorithm demonstrates how reducing the number of expensive multiplication operations can improve the theoretical efficiency of large matrix computations.

Q5 – Upgrading Closest Pair Using Divide and Conquer

Given n points in a plane, find the pair with the minimum Euclidean distance. The brute-force method checks every pair and takes $\Theta(n^2)$. The Divide-and-Conquer method sorts points, solves left and right halves, and checks a narrow strip near the dividing line.

Decomposition and combine operation

- Sort points by x-coordinate and retain an ordering by y-coordinate.
- Base case: for at most three points, check all pairs directly.
- Divide at the median x-coordinate into left and right point sets.
- Recursively obtain the closest distance in each half; let δ be the smaller distance.
- Build a vertical strip containing points whose horizontal distance from the dividing line is less than δ.
- Examine strip points in y-order. A geometric packing argument bounds the number of relevant following points, so the algorithm does not compare each strip point with every other point.

Pseudocode:

```
CLOSEST-PAIR(P):
    sort P by x; also prepare y-sorted order
    return CLOSEST(Px, Py)


CLOSEST(Px, Py):
    if number of points <= 3:
        return brute_force_closest_pair(Px)
    split Px into left and right at median x
    partition Py into corresponding leftY and rightY
    dL = CLOSEST(leftX, leftY)
```

```
    dR = CLOSEST(rightX, rightY)
    delta = min(dL, dR)
    strip = points in Py with abs(x - dividing_x) < delta
    best = delta
    for each strip point in y-order:
        compare it with only the next at most 7 strip points
        update best distance
    return best
```

Python implementation

```
from math import hypot
def closest_pair_dc(points):
    px = sorted(points)
    py = sorted(points, key=lambda p: p[1])
    def brute(P):
        best = float("inf")
        for i in range(len(P)):
            for j in range(i + 1, len(P)):
                best = min(best, hypot(P[i][0] - P[j][0],
                                       P[i][1] - P[j][1]))
        return best
    def solve(X, Y):

        n = len(X)

        if n <= 3:
```

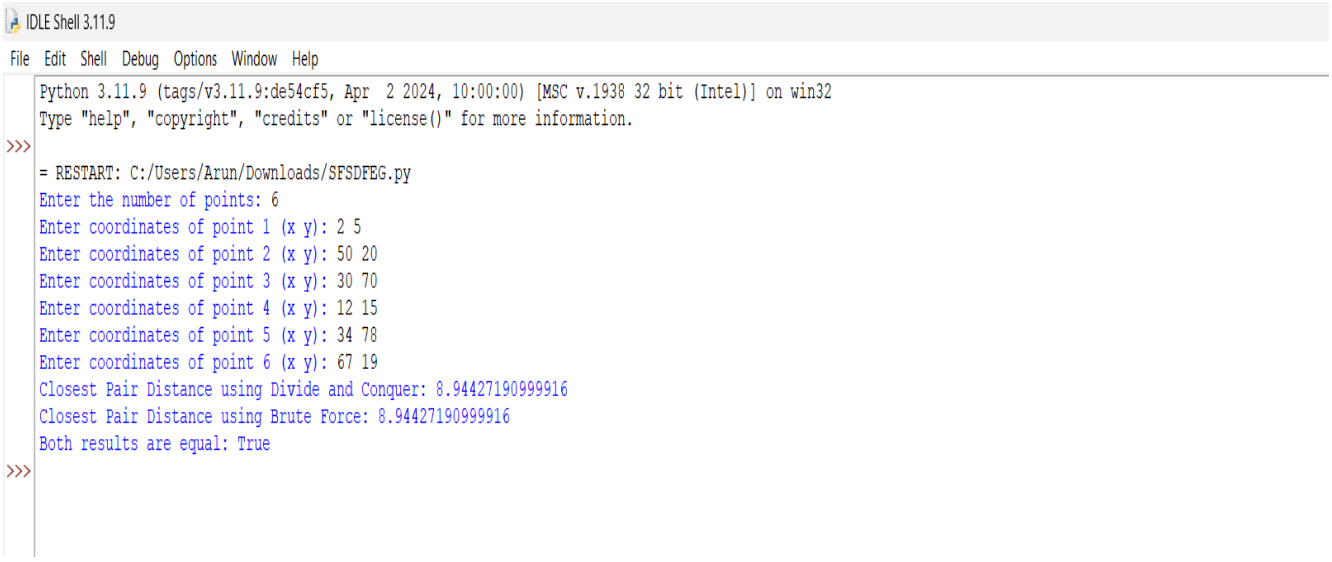
```
            return brute(X)


        mid = n // 2
        leftX, rightX = X[:mid], X[mid:]
        split_x = X[mid][0]
        left_set = set(leftX)
        leftY = [p for p in Y if p in left_set]
        rightY = [p for p in Y if p not in left_set]
        d = min(solve(leftX, leftY), solve(rightX, rightY))
        strip = [p for p in Y if abs(p[0] - split_x) < d]
        best = d
        for i in range(len(strip)):
            for j in range(i + 1, min(i + 8, len(strip))):
                best = min(best, hypot(strip[i][0] - strip[j][0],
                                       strip[i][1] - strip[j][1]))
        return best
    if len(points) < 2:
        return float("inf")
    return solve(px, py)
def closest_pair_bruteforce(points):
    best = float("inf")
    for i in range(len(points)):
        for j in range(i + 1, len(points)):
            best = min(best, hypot(points[i][0] - points[j][0],
                                   points[i][1] - points[j][1]))


    return best
```

```
# Getting input from the user
n = int(input("Enter the number of points: "))
if n < 2:
    print("Please enter at least 2 points.")
else:
    pts = []
    for i in range(n):
        x, y = map(float, input(f"Enter coordinates of point {i + 1} (x y): ").split())
        pts.append((x, y))
    dc_result = closest_pair_dc(pts)
    brute_result = closest_pair_bruteforce(pts)
    print("Closest Pair Distance using Divide and Conquer:", dc_result)
    print("Closest Pair Distance using Brute Force:", brute_result)
    print("Both results are equal:", abs(dc_result - brute_result) < 1e-9)
```

OUTPUT:



Implementation note: the reference code assumes distinct point tuples when using a set to partition y-order. For datasets containing duplicate coordinate tuples, use point IDs rather than tuple values. A

production implementation should also count distance evaluations and avoid rebuilding sets at every recursion level.

Recurrence and complexity

The standard efficient implementation has $T(n) = 2T(n/2) + \Theta(n)$, because the two recursive halves are solved and the y-ordered strip is processed in linear time using the bounded-neighbour property. Therefore $T(n) = \Theta(n \log n)$. Initial sorting is also $O(n \log n)$. The brute-force baseline takes $\Theta(n^2)$ distance calculations.

Why the strip does not compare every pair

Within each half, points are already separated by at least δ. In the strip, geometric packing limits how many points can lie close enough to a given point to improve the current best distance. When points are scanned in y-order, only a constant number (commonly bounded by seven in the standard 2D proof) of following points need to be checked. This is why the combine stage remains linear.

Experimental comparison

Create the same point datasets for both algorithms: uniformly random points, clustered points, points on a grid, and points with a very close pair across the dividing line. Use n = 100, 500, 1,000, 2,000, 5,000 and 10,000. Record the minimum distance, distance evaluations and median runtime over repeated runs.

n	Brute-force pair checks $n(n-1)/2$	D&C expected growth	Measured D&C distance evaluations
100	4,950	$O(n \log n)$	Run and record
500	124,750	$O(n \log n)$	Run and record
1,000	499,500	$O(n \log n)$	Run and record
2,000	1,999,000	$O(n \log n)$	Run and record

5,000	12,497,500	O(n log n)	Run and record

D&C becomes practically advantageous when the cost of sorting and recursive bookkeeping is outweighed by the avoided pairwise distance calculations. The crossover depends on the implementation and hardware; report the measured crossover rather than claiming a universal value.

Conclusion:

The Closest Pair of Points problem demonstrates how geometric problems can be solved efficiently using the Divide and Conquer approach. The objective is to identify two points having the minimum Euclidean distance among a given set of two-dimensional points. The algorithm sorts the points based on their X and Y coordinates and divides them into two approximately equal groups. The closest pair is calculated independently in both groups, and the smaller distance is selected. A narrow strip around the dividing line is then examined because a closer pair may contain points from different groups. By checking only a bounded number of neighbouring points in the strip, unnecessary distance calculations are reduced. The algorithm achieves $\Theta(n \log n)$ time complexity, whereas the Brute Force approach requires $\Theta(n^2)$ time because it checks every possible pair. The implementation compares the results of both approaches to verify the correctness of the calculated minimum distance. This problem provides practical understanding of geometric computation, sorting, recursive decomposition, and efficient combination of results.

Q6 – Advanced Divide-and-Conquer Mini-Investigations

Student A – Median of Medians

Selection finds the k-th smallest element without fully sorting the array. Median of Medians guarantees a sufficiently good pivot by grouping elements into small groups, finding each group's median, and recursively selecting the median of those medians.

Grouping and pivot selection

- Divide the input into groups of five (the final group may contain fewer elements).
- Sort each small group and take its median.
- Recursively select the median of the group medians as the pivot.
- Partition the full array around the pivot.
- Recurse only into the side containing the requested rank; return the pivot if its rank is k.

Pseudocode:

```
SELECT(A, k):
    if length(A) <= 5:
        sort A
        return A[k-1]
    divide A into groups of at most 5
    medians = median of each sorted group
    pivot = SELECT(medians, ceil(length(medians)/2))
    partition A into L (< pivot), E (= pivot), G (> pivot)
    if k <= length(L):
        return SELECT(L, k)
    if k <= length(L) + length(E):
        return pivot
```

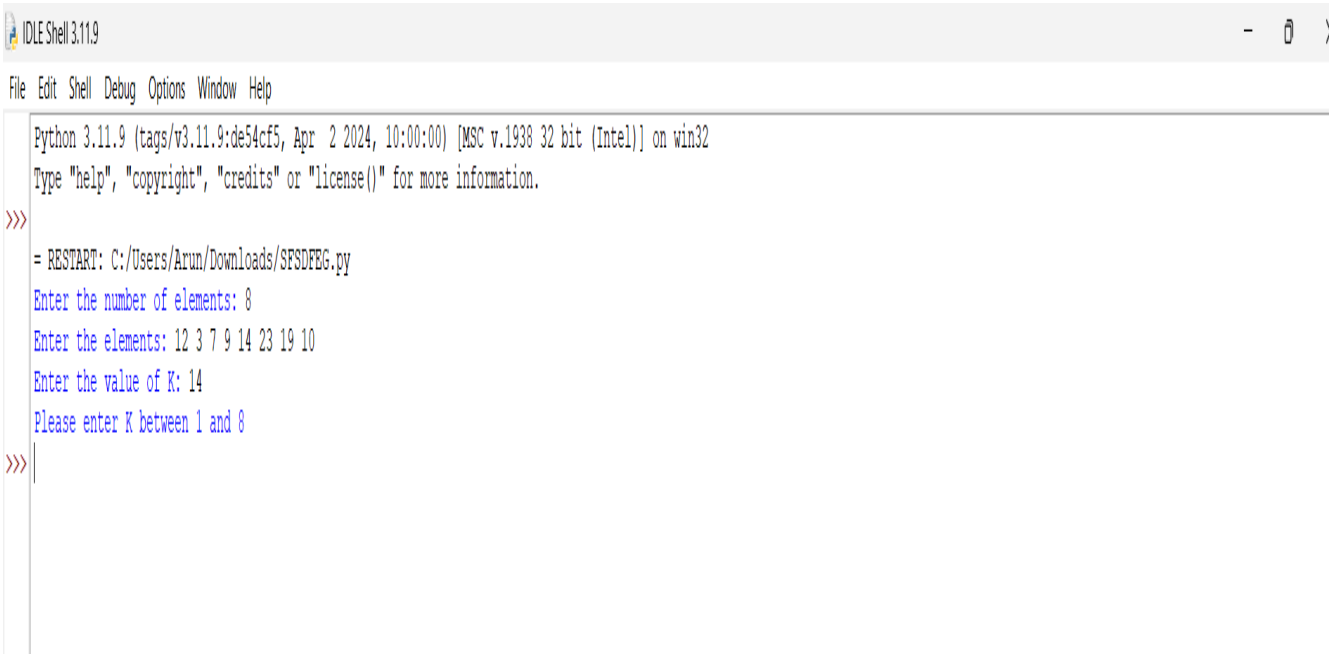
```
        return SELECT(G, k - length(L) - length(E))
```

Python implementation:

```
def median_of_medians_select(a, k)
    if not 1 <= k <= len(a):
        raise ValueError("k must be between 1 and n")
    if len(a) <= 5:
        return sorted(a)[k - 1]
    groups = [a[i:i + 5] for i in range(0, len(a), 5)]
    medians = [sorted(g)[len(g) // 2] for g in groups]
    pivot = median_of_medians_select(medians, (len(medians) + 1) // 2)
    low = [x for x in a if x < pivot]
    equal = [x for x in a if x == pivot]
    high = [x for x in a if x > pivot]
    if k <= len(low):
        return median_of_medians_select(low, k)
    if k <= len(low) + len(equal):
        return pivot
    return median_of_medians_select(high, k - len(low) - len(equal))
# Getting input from the user
n = int(input("Enter the number of elements: "))
if n <= 0:
    print("Please enter a valid number of elements.")
else:
    data = list(map(int, input("Enter the elements: ").split()))
```

```
    if len(data) != n:
        print("Please enter exactly", n, "elements.")
    else:
        k = int(input("Enter the value of K: "))
        if 1 <= k <= n:
            result = median_of_medians_select(data, k)
            print("The", k, "th smallest element is:", result)
        else:
            print("Please enter K between 1 and", n)
```

OUTPUT:



Worst-case guarantee and recurrence:

The pivot construction guarantees that a constant fraction of elements is discarded in the worst case. A standard upper-bound recurrence is $T(n) \leq T(\lceil n/5 \rceil) + T(7n/10 + O(1)) + O(n)$, which solves to $T(n) = O(n)$. Thus deterministic Median of Medians has worst-case linear selection time.

Comparison with QuickSelect

Selection method	Pivot choice	Expected / worst-case time	Comment
QuickSelect (random pivot)	Random element	Expected O(n); worst $O(n^2)$	Usually simple and fast; randomisation reduces risk but does not eliminate a bad run
Deterministic poor pivot	First/last element under adversarial order	Can be $O(n^2)$	Repeatedly leaves nearly all elements on one side
Median of Medians	Median of group medians	Worst-case O(n)	More work per pivot, but guarantees progress

Pivot quality is central because it determines how much of the array is discarded after each partition. A balanced pivot reduces the remaining problem substantially; a poor pivot can leave almost the entire input for the next recursive call.

Experiment design

Test random, sorted, reverse-sorted, and adversarial arrays at n = 100, 500, 1,000, 5,000 and 10,000. Compare selected values against sorted(A)[k−1]. Instrument partition sizes, comparisons and recursion depth. Demonstrate deterministic first-element QuickSelect on sorted data, randomized QuickSelect on repeated seeds, and Median of Medians. Randomisation reduces the likelihood of repeated poor partitions but cannot provide a deterministic worst-case guarantee.

Student B – Meet-in-the-Middle Subset Sum

Given n numbers and a target S, determine whether some subset sums to S. Straight enumeration considers 2^n subsets. Meet-in-the-Middle splits the numbers into two halves, enumerates subset sums for each half, and searches for complementary sums.

Decomposition and combine strategy

- Split the input into L and R, each approximately n/2 elements.
- Generate every subset sum of L and every subset sum of R.
- For each left sum x, search for S−x among the right sums.
- If a match exists, a subset with total S has been found.

Pseudocode:

```
SUBSET-SUM-MITM(A, target):
    split A into L and R
    leftSums = all subset sums of L
    rightSums = all subset sums of R
    store rightSums in a hash set
    for x in leftSums:
        if target - x is in rightSums:
            return TRUE
    return FALSE
```

Python implementation

```
def subset_sums(values):
    sums = [0]
```

```
    for v in values:
        sums += [x + v for x in sums]
    return sums
def subset_sum_mitm(a, target):
    mid = len(a) // 2
    left_sums = subset_sums(a[:mid])
    right_sums = set(subset_sums(a[mid:]))
    for x in left_sums:
        if target - x in right_sums:
            return True
    return False
# Getting input from the user
n = int(input("Enter the number of elements: "))

if n <= 0:
    print("Please enter a valid number of elements.")
else:
    data = list(map(int, input("Enter the elements: ").split()))
    if len(data) != n:
        print("Please enter exactly", n, "elements.")
    else:
        target = int(input("Enter the target sum: "))
        result = subset_sum_mitm(data, target)
        if result:
            print("Subset with the given target sum exists.")
        else:
            print("Subset with the given target sum does not exist.")
```

OUTPUT:



Search-space and complexity analysis:

Each half has about 2^(n/2) subsets. Generating and searching the sums takes O(2^(n/2)) expected time with hashing, and O(2^(n/2)) additional memory. This is exponentially smaller in time than straightforward O(2^n) enumeration, but it uses substantial storage. Sorting one list of sums and using binary search is an alternative with O(2^(n/2) log 2^(n/2)) sorting/search work.

n	**Straight enumeration 2^n**	**MITM sums per half ≈ 2^(n/2)**	**Memory trend**
20	1,048,576	1,024	Manageable
30	1,073,741,824	32,768	Higher
40	1,099,511,627,776	1,048,576	Substantial
50	1,125,899,906,842,624	33,554,432	Very high

Meet-in-the-Middle reduces computational work by replacing one search over 2^n combinations with two lists of approximately 2^(n/2) partial results. The trade-off is memory: storing these partial sums can become the limiting factor.

Conclusion:

Median of Medians and Meet-in-the-Middle introduce advanced techniques for solving selection and subset-based problems efficiently. Median of Medians is used to identify the Kth smallest element in an unsorted array. It divides the array into smaller groups, usually containing five elements each, calculates the median of every group, and selects the median of these medians as a pivot. The original array is partitioned based on this pivot, and the required partition is processed recursively until the Kth smallest element is found. This technique guarantees worst-case linear time complexity of O(n). Meet-in-the-Middle is used to solve problems involving subsets and combinations, particularly the Subset Sum problem. It divides the input array into two approximately equal halves and generates all possible subset sums independently for both halves. The generated results are stored and compared to determine whether a combination of subset sums produces the required target value. This reduces the search effort to approximately O(2^(n/2)) time with comparable memory requirements for the standard subset-generation approach. These techniques demonstrate how selecting an appropriate strategy for a particular problem can reduce computational effort and improve efficiency.

Q7 – Day-4 Design Challenge: Design Divide and Conquer from First Principles

Chosen challenge problem: Count inversions in an array

The supplied assignment brief describes the six design stages but does not name the actual Day-4 challenge problem. This report uses the following proposed challenge: count the number of inversions in an array. An inversion is a pair of indices (i,j) such that i < j and A[i] > A[j]. Confirm with the faculty that this is the intended problem before submission if a different challenge was announced.

Example: for A = [2, 4, 1, 3, 5], the inversions are (2,1), (4,1), and (4,3), so the count is 3.

Stage 1 – Independent design proposals

Student A – Initial proposal (independent)

- Decomposition: split the array into left and right halves until each part contains one element.
- Base case: an array of size zero or one has zero inversions.
- Subproblem information: each recursive call returns a sorted version of its section and the number of inversions within that section.
- Combine strategy: merge the two sorted halves and count cross-inversions when an element from the right half is placed before remaining elements in the left half.

Student B – Initial proposal (independent)

- Decomposition: divide the index range into two contiguous halves so pairs are either fully in the left half, fully in the right half, or cross the split.
- Base case: one item cannot form an inversion.
- Subproblem information: return the sorted values and the inversion count for that half.
- Combine strategy: use a two-pointer merge. When the next right value is smaller than the next left value, add the number of unmerged left values to the cross-inversion count.

The above are proposed example records. Before submitting, each student should edit this section to reflect the design they actually wrote independently before discussion. Do not claim these were independently produced if the process did not occur.

Stage 2 – Compare the two designs

Comparison point	Student A proposal	Student B proposal	Joint decision
Decomposition	Split array recursively	Split contiguous index ranges	Same recursive split is retained
Base case	0 or 1 item	One item	Use length ≤ 1
Returned information	Sorted subarray and internal count	Sorted subarray and internal count	Retain both
Combine	Count while merging	Add remaining-left count when right item is smaller	Use two-pointer merge counting
Strength	Connects counting to merge operation	Clearly explains cross-half pairs	Combine both explanations
Potential issue	Must count all remaining left values, not just one	Equal values must not count as inversions	Use strict comparison: left > right

Stage 3 – Joint final design

Divide the array into two halves. Recursively count inversions in each half and return each half in sorted order. During merge, if left[i] ≤ right[j], copy left[i]. Otherwise right[j] is smaller than every remaining element in the left half, so add len(left)−i to the count and copy right[j]. Add left and right internal counts to the cross count.

Pseudocode:

```
COUNT-INVERSIONS(A):
    if length(A) <= 1:
        return (A, 0)
    mid = floor(length(A)/2)
    (Lsorted, leftCount) = COUNT-INVERSIONS(A[0:mid])
    (Rsorted, rightCount) = COUNT-INVERSIONS(A[mid:])
    (merged, crossCount) = MERGE-COUNT(Lsorted, Rsorted)
    return (merged, leftCount + rightCount + crossCount)


MERGE-COUNT(L, R):
    i = 0; j = 0; count = 0; output = []
    while i < length(L) and j < length(R):
        if L[i] <= R[j]:
            append L[i]; i = i + 1
        else:
            append R[j]
            count = count + (length(L) - i)
            j = j + 1
    append remaining elements of L and R
    return (output, count)
```

Python implementation and brute-force baseline:

```
def merge_count(L, R):
```

```
    i = 0
    j = 0
    count = 0
    output = []

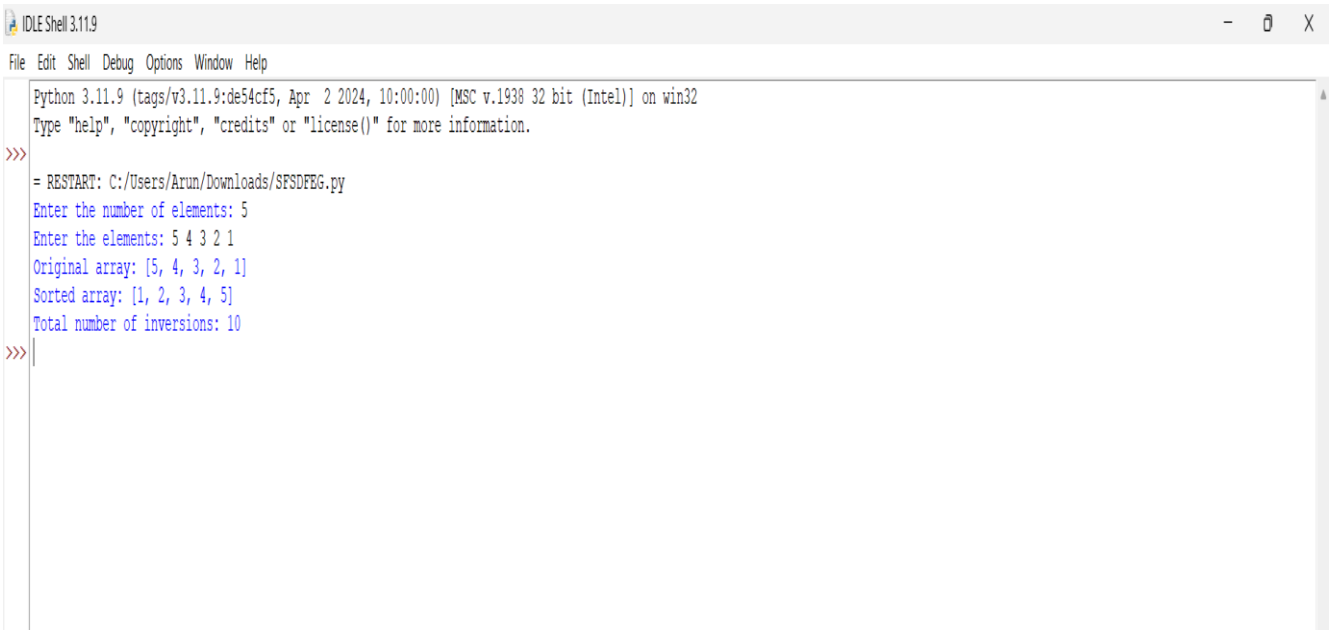
    while i < len(L) and j < len(R):
        if L[i] <= R[j]:
            output.append(L[i])
            i += 1
        else:
            output.append(R[j])
            count += len(L) - i
            j += 1
    output.extend(L[i:])
    output.extend(R[j:])
    return output, count

def count_inversions(A):
    if len(A) <= 1:
        return A, 0
    mid = len(A) // 2
    Lsorted, leftCount = count_inversions(A[:mid])
    Rsorted, rightCount = count_inversions(A[mid:])
    merged, crossCount = merge_count(Lsorted, Rsorted)
    return merged, leftCount + rightCount + crossCount
# Getting input from the user
```

```
n = int(input("Enter the number of elements: "))


if n <= 0:
    print("Please enter a valid number of elements.")
else:
    data = list(map(int, input("Enter the elements: ").split()))
    if len(data) != n:
        print("Please enter exactly", n, "elements.")
    else:
        sorted_data, inversions = count_inversions(data)
        print("Original array:", data)
        print("Sorted array:", sorted_data)
        print("Total number of inversions:", inversions)
```

OUTPUT:



Stage 4 – Mathematical validation (Student A primary)

The recurrence is $T(n) = 2T(n/2) + \Theta(n)$. The two recursive calls process halves of size n/2. The merge-and-count operation scans each element at most once, so its work is $\Theta(n)$. By the Master Theorem, $T(n) = \Theta(n \log n)$. The brute-force baseline checks all index pairs and takes $\Theta(n^2)$ time.

Stage 4 – Experimental validation (Student B primary)

Test arrays of lengths 10, 100, 500, 1,000, 5,000 and 10,000. Include sorted ascending (zero inversions), reverse sorted ($n(n-1)/2$ inversions), random arrays, arrays with duplicates, empty and single-element arrays, and arrays containing negative values. For every test, verify that the Divide-and-Conquer count equals the brute-force count for sizes small enough to run the baseline.

Test case	Input example / pattern	Expected inversion count
Already sorted	[1,2,3,4,5]	0
Reverse sorted	[5,4,3,2,1]	10
Mixed	[2,4,1,3,5]	3
Duplicates	[2,2,1]	2
Single element	[7]	0
Empty	[]	0

Stage 5 – Cross-defence

Student A should explain the evidence: correctness matches the brute-force baseline on all small test cases, while comparison/time growth is much slower for the quadratic baseline as n increases. Student B should explain the recurrence: two calls on n/2 elements plus one linear merge-and-count pass gives $2T(n/2)+\Theta(n)$.

Stage 6 – Joint evaluation

- Why suitable for Divide and Conquer: every inversion belongs entirely to the left half, entirely to the right half, or crosses the split; these categories can be counted without double-counting.
- Work reduced: pairwise comparison of all index pairs is replaced by recursive sorting and a linear merge pass.
- Additional costs: recursion stack and temporary merge arrays require O(n) auxiliary storage.
- When D&C may not be appropriate: for very small arrays, the simple brute-force method may be easier and have lower overhead; streaming inputs or strict memory limits may also favour another design.
- Algorithm-selection lesson: look for independent smaller subproblems, a base case, a way to combine compact summaries, and a combine step whose cost does not erase the benefit of decomposition.

Conclusion:

Counting Inversions demonstrates how the Divide and Conquer technique can be applied to solve a problem involving the identification of incorrectly ordered element pairs in an array. An inversion is a pair of elements in which the earlier element is greater than the later element. The Brute Force method checks every possible pair to calculate the total number of inversions, resulting in $\Theta(n^2)$ time complexity. The Divide and Conquer approach improves this process by dividing the array into two smaller subarrays and recursively counting inversions within each subarray. The sorted subarrays are then merged while counting cross inversions between them. Whenever an element from the right subarray is smaller than the current element in the left subarray, all the remaining elements in the left subarray form inversions with that element. This allows multiple inversions to be counted efficiently during the merging process. The recurrence relation is $T(n) = 2T(n/2) + O(n)$, resulting in an overall time complexity of $\Theta(n \log n)$. The implementation also produces the sorted array along with the total

inversion count. This problem demonstrates how the Merge Sort technique can be extended to solve additional computational problems and how efficient algorithm design reduces unnecessary operations.

Student contribution record

The following primary responsibilities are taken from the assignment brief. The shared-work column must be completed to reflect the actual joint work. Both students remain responsible for understanding the entire assignment.

Question	Student A – primary contribution	Student B – primary contribution	Joint work
Q1	Max/Min decomposition, combine logic and comparison analysis; Binary Search experiments	Max/Min datasets and comparison instrumentation; Binary Search elimination logic	Design, implement and test both algorithms
Q2	Quick Sort balanced/unbalanced experiments, partitions and recursion depth	Merge Sort analysis and comparison with Quick Sort	Implement both sorts, choose pivot strategy, run experiments and answer joint design question
Q3	Karatsuba derivation, recurrence and asymptotic analysis	Crossover experiment, timings, multiplication counts and overhead analysis	Implement standard and Karatsuba multiplication; correctness and joint conclusion
Q4	Matrix-size experiments,	Block algebra, seven products, correctness	Implement and verify conventional and

	multiplication counts, timings and engineering analysis	and padding explanation	Strassen multiplication; decide practical use
Q5	D&C decomposition, recursive returns, combine and boundary explanation	Equivalent datasets, distance counts, runtime comparison and crossover investigation	Implement and compare brute-force and D&C algorithms
Q6	Median of Medians implementation, grouping, pivot and worst-case guarantee; MITM verification	Meet-in-the-Middle implementation, subset sums, time-memory analysis; pivot-quality verification	Review both investigations and cross-explain
Q7	Independent proposal; recurrence formulation, solution and mathematical validation	Independent proposal; implementation, correctness tests and experimental validation	Compare proposals, create final design, cross-defend and jointly evaluate